

Deep Learning

4. Loss functions, scaling, regularization

Romain Tavenard

Optimization in general

- **Goal:** Tune model parameters so as to minimize error
- Measuring error
 - Through a cost function / loss function
 - Typical example: Mean Squared Error in regression settings

Loss functions

- To train a model, we need to **measure how wrong it is**:
that's the loss function $\mathcal{L}$
- Standard losses
 - **Mean Squared Error (MSE)** for regression
(penalizes large errors)

$$\mathcal{L}(x_i, y_i; \theta) = (m(x_i; \theta) - y_i)^2$$

- **Cross-entropy** for classification
(penalizes confident wrong predictions)

$$\mathcal{L}(x_i, y_i; \theta) = -\log P_{\theta}(y = y_i \mid x_i)$$

- Technical requirement for gradient descent:
 $\mathcal{L}$ must be differentiable

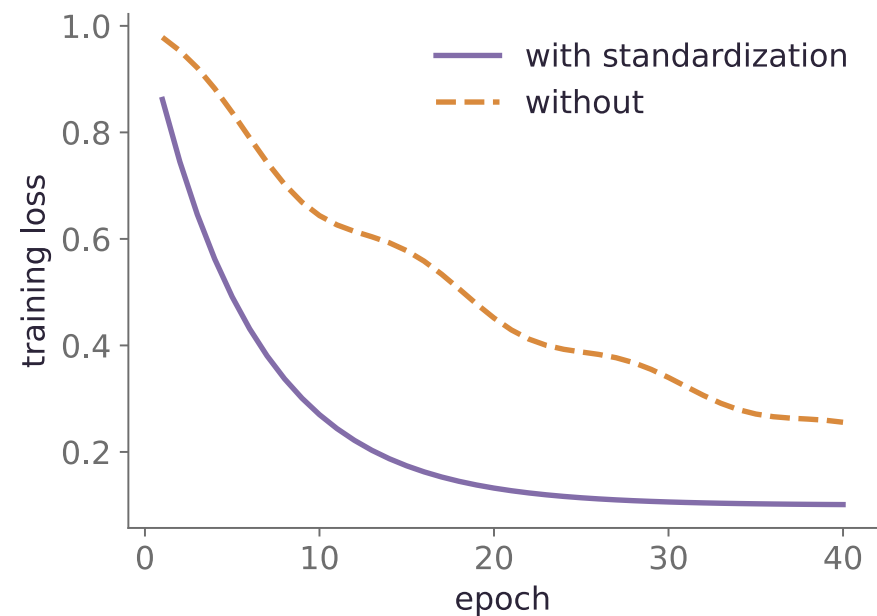
Scaling

- In practice, **scaling input features** is critical for stable training

- ▶ Recommendation: center-reduce each feature

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}$$

- ▶ or use domain-specific scaling
(e.g. images in $[0, 1]$)



- **Intuition:** keep signal “loudness” (variance) **constant** across layers
 - too large $\rightarrow$ activations explode
 - too small $\rightarrow$ fade to 0 (vanishing gradients, *cf* next course)
- **Where numbers come from:** neuron sums n_{in} inputs $\rightarrow$ variance $\times n_{\text{in}}$ $\rightarrow$ to keep it stable:

$$\text{Var}(w) = \frac{1}{n_{\text{in}}}$$

Two classic schemes = refinements of $\text{Var}(w) = 1/n_{\text{in}}$:

- **Xavier / Glorot** (tanh, sigmoid):

$$w \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

- ▶ stable forward (n_{in}) **and** backward (n_{out}) $\rightarrow$ harmonic-mean
- ▶ assumes φ linear near 0

- **He** (ReLU):

$$w \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

- ▶ ReLU kills **half** the signal $\rightarrow$ **double** the variance (factor 2)

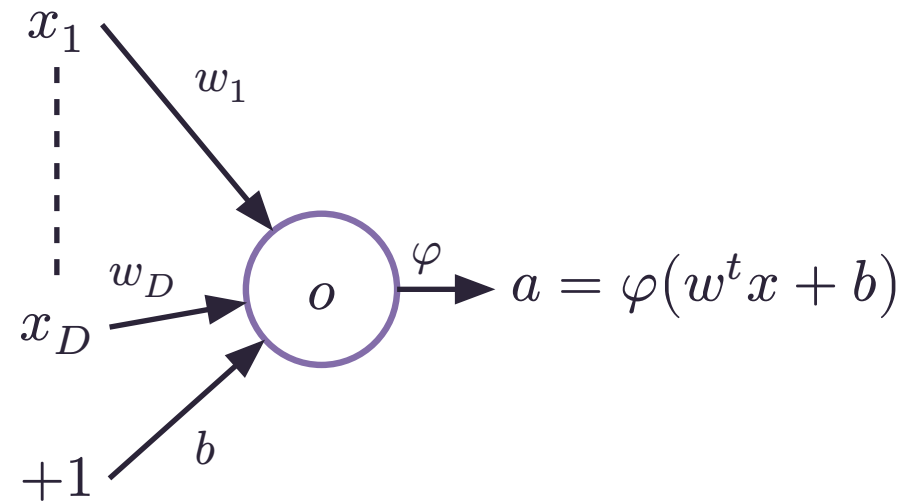
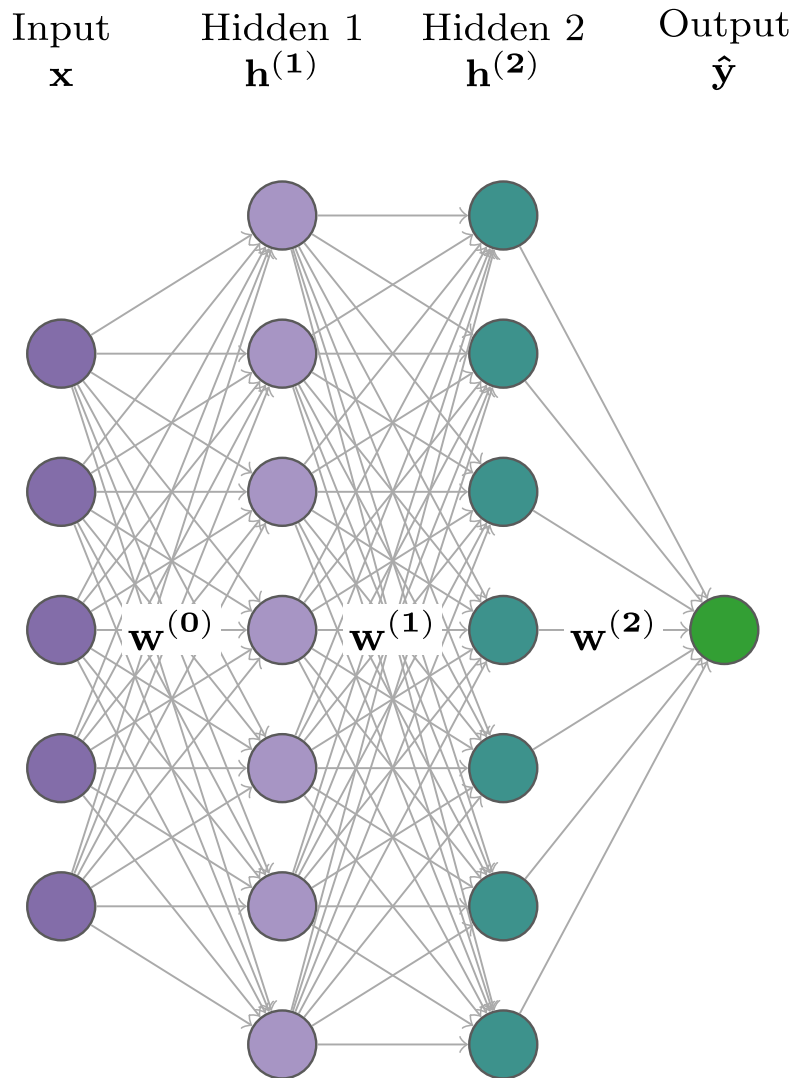
- **Init fixes scale only at $t = 0$** $\rightarrow$ drifts as weights update
- **Batch Normalization:** Same goal as init, **enforced during training**: re-standardize layer inputs **every** mini-batch

$$\hat{z}^{(l)} = \frac{z^{(l)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}$$

- then learnable scale γ + shift β
- **Train vs eval mode:**
 - **Train:** $\mu_{\mathcal{B}}, \sigma_{\mathcal{B}}^2$ = statistics of the **current mini-batch**
 - **Eval:** mini-batch stats unreliable (batch size 1, no batch)
 $\rightarrow$ **running average** μ, σ^2 from training, kept **fixed** at test time

Vanishing gradients

Neural networks and back-propagation



$$\frac{\partial a^{(l)}}{\partial o^{(l)}} = \varphi'(o^{(l)})$$

$$\frac{\partial o^{(l)}}{\partial a^{(l-1)}} = w^{(l-1)}$$

Neural networks and back-propagation

Vanishing gradients

$$\frac{\partial \mathcal{L}}{\partial w^{(2)}} = \frac{\partial \mathcal{L}}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial w^{(2)}}$$

$$\frac{\partial \mathcal{L}}{\partial w^{(1)}} = \frac{\partial \mathcal{L}}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial o^{(2)}} \frac{\partial o^{(2)}}{\partial w^{(1)}}$$

$$\frac{\partial \mathcal{L}}{\partial w^{(0)}} = \frac{\partial \mathcal{L}}{\partial a^{(3)}} \underbrace{\frac{\partial a^{(3)}}{\partial o^{(3)}}}_{\varphi'(o^{(3)})} \frac{\partial o^{(3)}}{\partial a^{(2)}} \underbrace{\frac{\partial a^{(2)}}{\partial o^{(2)}}}_{\varphi'(o^{(2)})} \frac{\partial o^{(2)}}{\partial a^{(1)}} \underbrace{\frac{\partial a^{(1)}}{\partial o^{(1)}}}_{\varphi'(o^{(1)})} \frac{\partial o^{(1)}}{\partial w^{(0)}}$$

Deeper networks and vanishing gradients

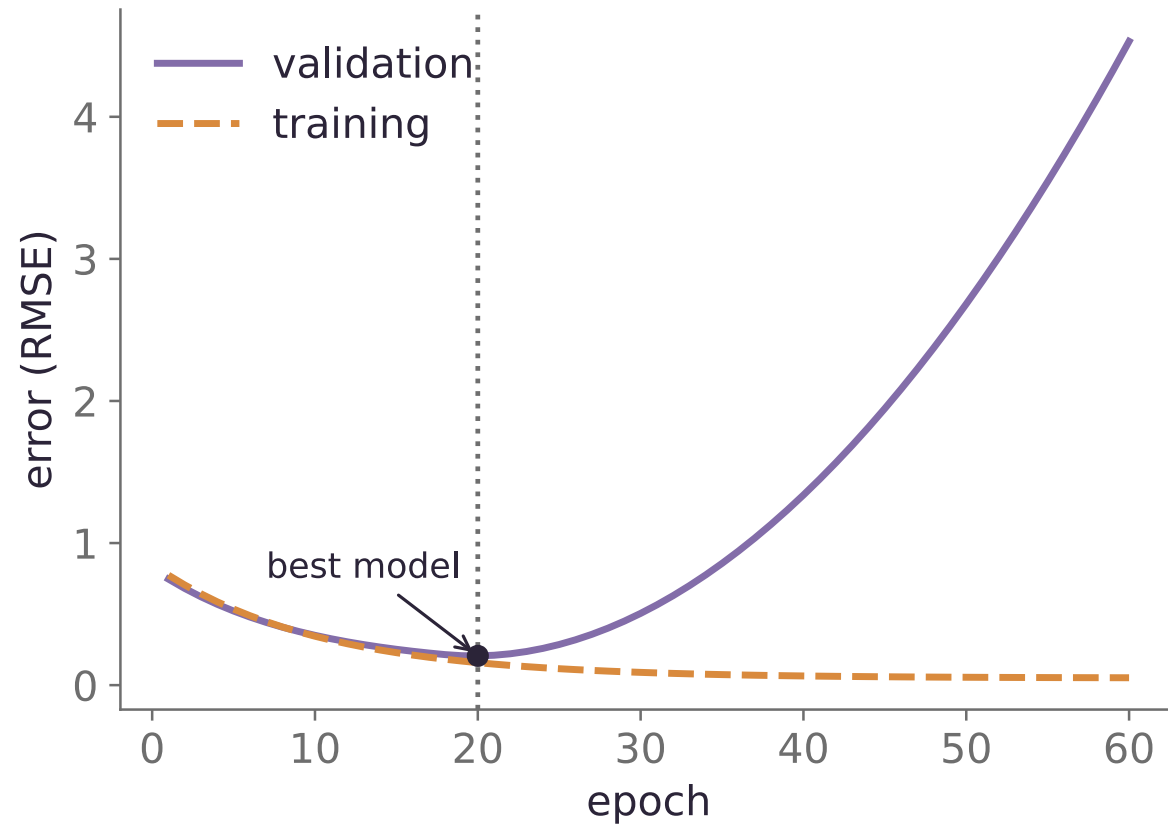
- Deeper networks = higher-level understanding
- But a problem appears: **vanishing gradients**
- **Intuition:**
 - gradient travels back through **every** later layer
 - one multiplicative factor per layer
 - many factors < 1
 - product shrinks to 0
 - early layers barely learn
 - **ReLU helps:** $\varphi' = 1$ (active side) → factor 1, no shrinking

Regularization

- Optimization (SGD) to minimize a loss function
 - Larger & deeper nets improve (training) performance
 - Risks over-fitting

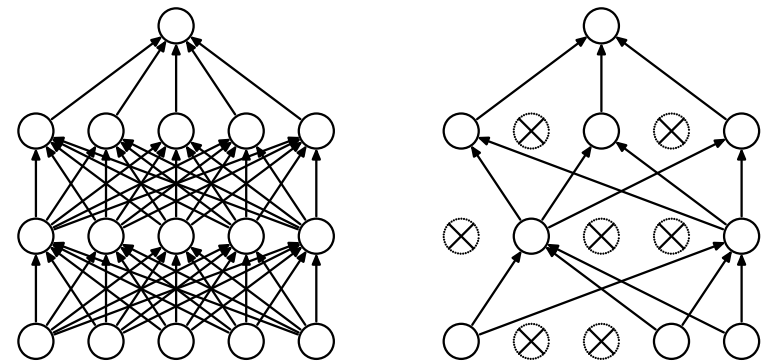
$$\arg \min_{\theta} \sum_{(x_i, y_i) \in \mathcal{D}_t} \mathcal{L}(x_i, y_i; \theta) \neq \arg \min_{\theta} \mathbb{E}_{x, y \sim \mathcal{D}} \mathcal{L}(x, y; \theta)$$

- Regularization tricks
 - Early stopping (cf. Gradient boosting)
 - Dropout (relates to Random Forests)



Stop where validation error is lowest

- At each mini-batch, randomly **switch off** a fraction of neurons
- All neurons are eventually trained over the full process
- Similar in spirit to random forests (feature subsampling)



Dropout illustration — left: full network; right: same network with 40% of neurons switched off for one mini-batch.

Source: [Srivastava et al., 2014]

- Loss functions
 - Mean Squared Error for regression
 - Logistic loss for classification
- Keep activations well-scaled
 - input standardization
 - init $\rightarrow$ scale at start
 - BatchNorm $\rightarrow$ scale during training
- Vanishing gradients
 - deeper networks $\rightarrow$ product shrinks to 0
 - ReLU helps (factor = 1 on active side)
- Regularization
 - Early stopping
 - DropOut